Aula 21: Diagrama de Classes e Diagrama de Atividades

Fundamentação Estrutural e Comportamental baseada nos Capítulos 4 e 10 (Gilleanes Guedes)

Prof. Reginaldo Pereira Fernandes
Técnico em Informática para Internet

Instituto Federal do Ceará – Campus Tauá

27 de outubro de 2026

Sumário da Aula

1. Aprofundamento no Diagrama de Classes (Cap. 4)
2. Diagrama de Atividades: Fluxos e Processos (Cap. 10)
3. Atividade Prática: TauáDelivery Web (Classes e Atividades)

Do Caso de Uso à Estrutura Interna do Sistema

A Transição da Análise para o Projeto:

- Na **Aula 20 (Cap. 3)**, modelamos o sistema como uma *"caixa-preta"* através dos Casos de Uso.
- Na **Aula 21 (Cap. 4)**, abrimos a caixa para projetar o alicerce estático com o **Diagrama de Classes**.

Papel Central do Diagrama de Classes:

- É o diagrama estrutural mais importante da UML;
- Base direta para a geração de código-fonte OO e para o esquema de banco de dados.

Níveis de Modelagem (Guedes)

- **Modelo Conceitual:** Fase de análise; foca nas classes e atributos do domínio do problema.
- **Modelo de Domínio:** Fase de projeto; detalha assinaturas de métodos, navegabilidade e interfaces da solução.

Detalhamento de Atributos e Assinaturas de Operações

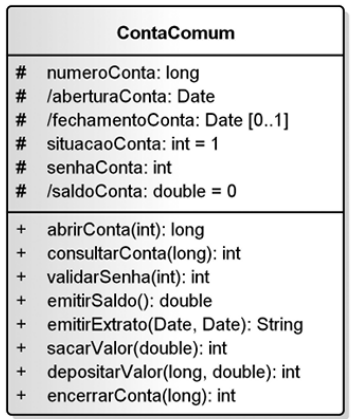
Sintaxe de Atributos (Fig. 4.3):

- `vis [/] nome: tipo [mult] = val {prop}`
- **Barra (/):** Indica **atributo derivado** (calculado dinamicamente, ex.: `/saldoConta`).
- **Multiplicidade** `[0..1]`: Indica atributo opcional (ex.: `/fechamentoConta`).
- **Valor Inicial:** Atribuído na instanciação (ex.: `situacaoConta: int = 1`).

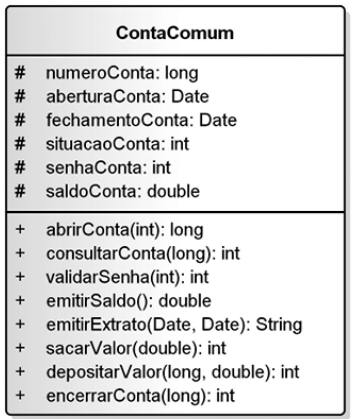
Assinatura de Operações (Fig. 4.2):

- Especifica parâmetros (`nome: Tipo`) e retorno : `TipoRetorno`.

Atributos Detalhados (Fig. 4.3):



Assinaturas de Métodos (Fig. 4.2):



Associação Unária (Reflexiva) e Definição de Papéis

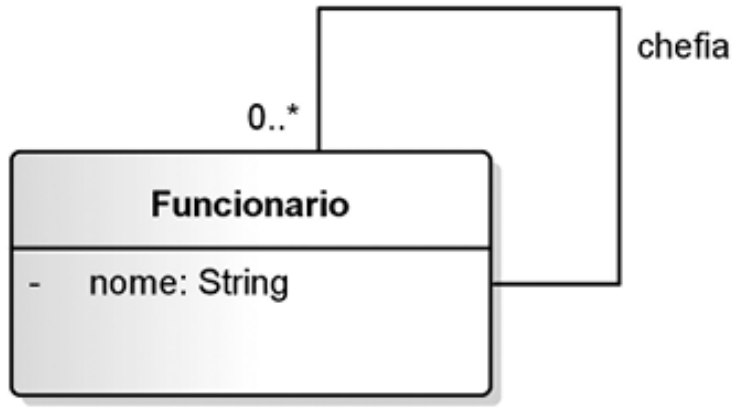
Associação Unária / Reflexiva (4.4):

- Ocorre quando instâncias de uma classe relacionam-se com outras instâncias da **mesma classe**.
- Exemplo clássico: um `Funcionario` gerencia vários outros objetos `Funcionario`.

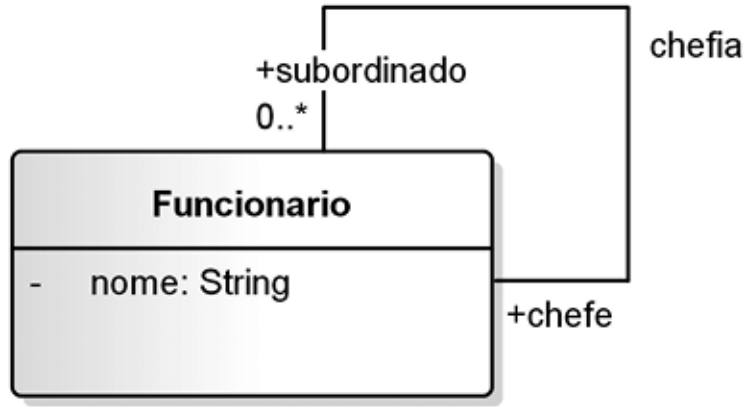
Papéis nas Extremidades (4.5):

- Esclarecem a função exercida por cada objeto no vínculo:
  - Extremidade `1`: papel `+chefe`;
  - Extremidade `*`: papel `+subordinados`.

Associação Reflexiva (Fig. 4.4):



Com Papéis Explícitos (Fig. 4.5):



Associação Binária, Multiplicidade e Navegabilidade

Associação Binária (4.6):

- Conecta objetos de duas classes distintas (ex.: um `Socio` registra `0..*` objetos `Dependente`).

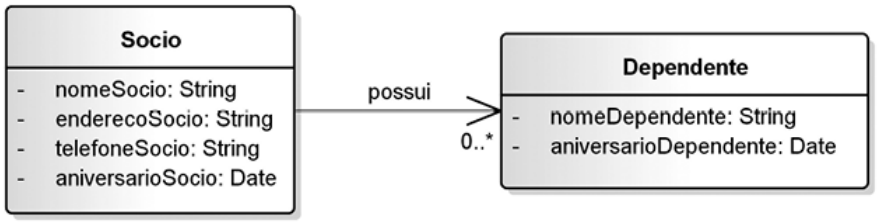
Navegabilidade (4.7):

- Representada por uma **seta aberta** indicando o sentido em que as mensagens e referências trafegam (`Socio` $\longrightarrow$ `Dependente`).

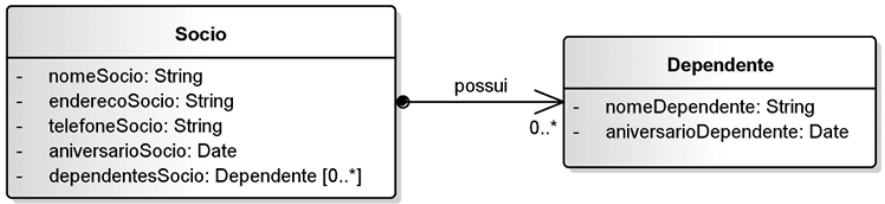
Fim de Associação Possuída (4.8):

- Pequeno ponto cheio (*class-owned end*) indicando que o papel torna-se um atributo de referência na classe origem.

Navegabilidade (Fig. 4.7):



Class-Owned End (Fig. 4.8):

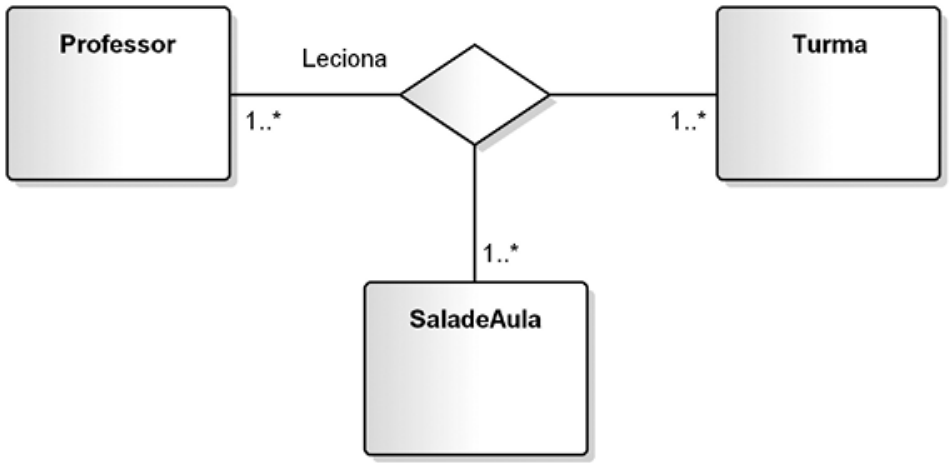


Associação Ternária ou N-ária

Quando utilizar?

- Conecta simultaneamente objetos de **três ou mais classes** através de um **losango centralizador**.
- Exemplo acadêmico (Fig. 4.9): o vínculo entre `Professor`, `Turma` e `SalaDeAula`.
- Um professor leciona para $1..*$ turmas utilizando $1..*$ salas (ex.: sala teórica e laboratório).

Associação Ternária (Figura 4.9):



Recomendação Prática

Use apenas quando o vínculo for indivisível entre as 3 classes; caso contrário, prefira uma classe intermediária.

Relacionamento Todo-Parte: Agregação (◊)

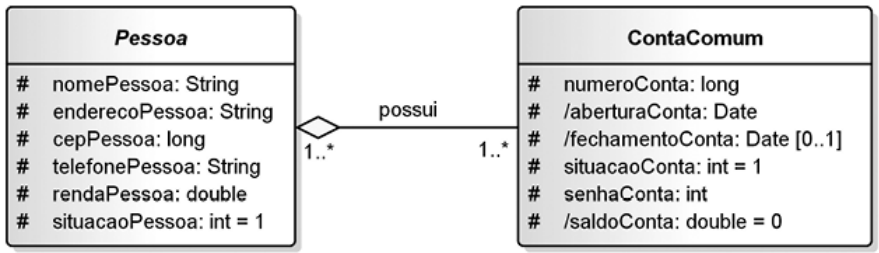
Conceito de Agregação:

- Demonstra que as informações de um **Objeto-Todo** são complementadas por um ou mais **Objetos-Parte**.
- Símbolo gráfico: **losango vazio (◊)** posicionado junto ao Objeto-Todo.

Independência de Ciclo de Vida:

- Vínculo mais fraco: os objetos-parte podem ser compartilhados e **continuam existindo** mesmo que o objeto-todo seja excluído.

Agregação na UML (Figura 4.10):



`Pessoa` (Todo) agrega `ContaComum` (Parte) em relação $1..*$ para $1..*$.

Relacionamento Todo-Parte Forte: Composição (♦)

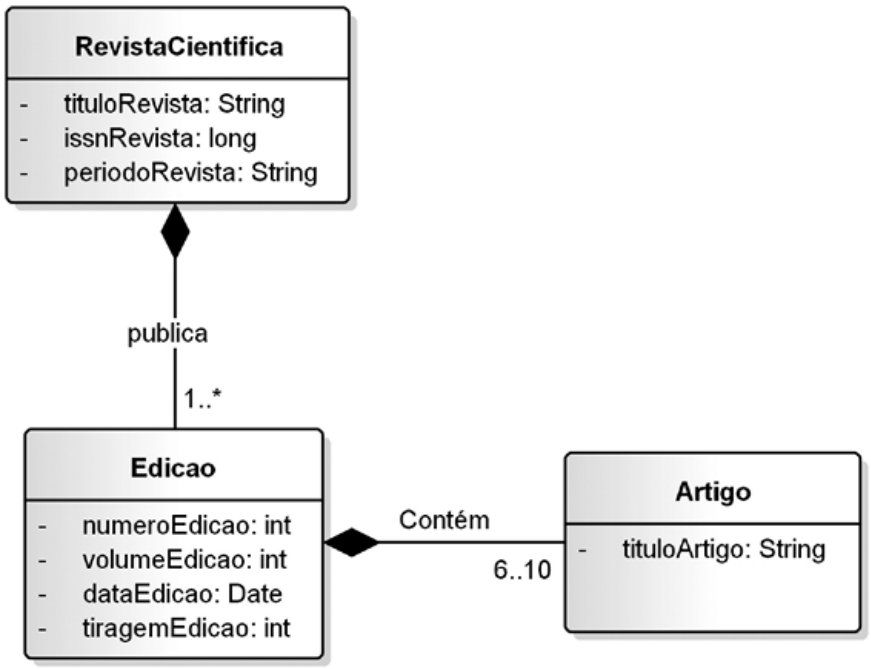
Conceito de Composição:

- Variação estrita da agregação com **vínculo forte e exclusivo** entre o Todo e suas Partes.
- Símbolo gráfico: **losango preenchido (♦)** ao lado do Objeto-Todo.

Regras Fundamentais:

1. Cada instância-parte pertence a **um único** objeto-todo por vez (multiplicidade 1 no lado do Todo);
2. Se o objeto-todo for destruído, todas as suas partes são **destruídas em cascata**!

Composição na UML (Figura 4.11):



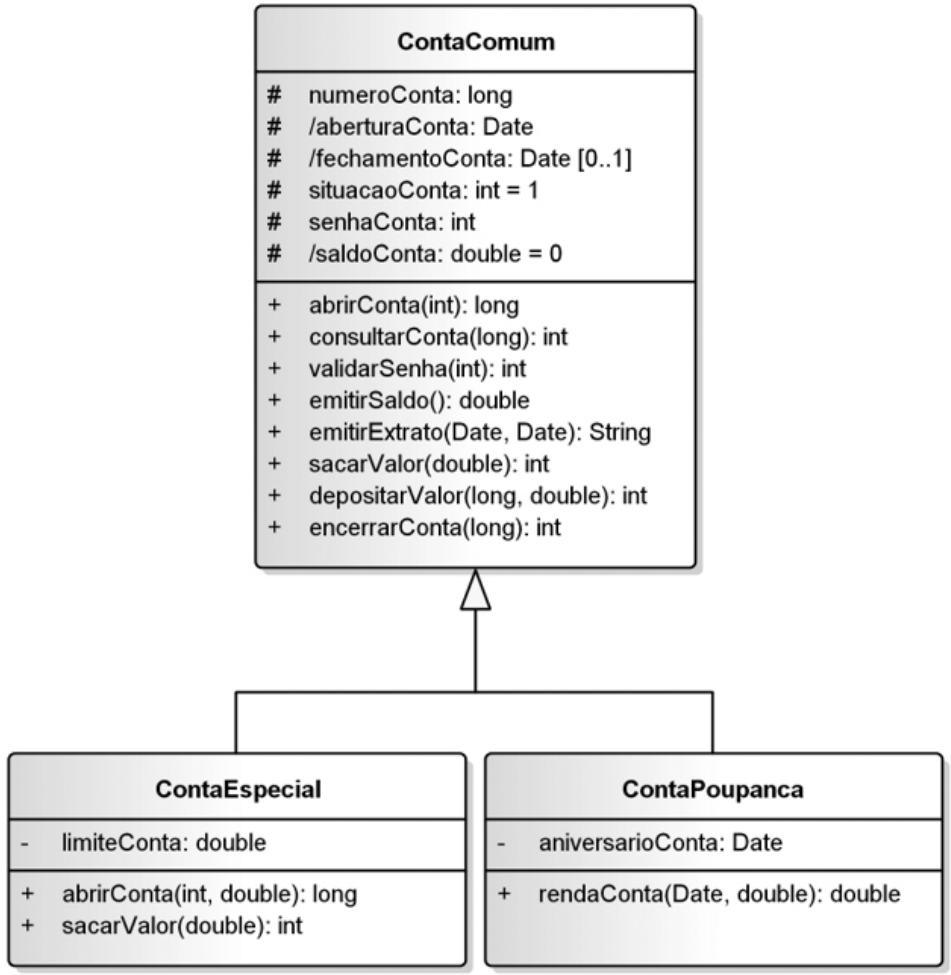
`RevistaCientifica` ♦ → `Edicao` ♦ → `Artigo`.

Generalização, Especialização e Classes Abstratas

Herança no Diagrama de Classes:

- Seta triangular vazia apontando da subclasse para a superclasse.
- **Classe Abstrata:** Não pode ser instanciada diretamente; seu nome é grafado em *itálico* (ex.: *Pessoa* na Fig. 4.12).
- **Classes Concretas:** Herdam atributos/métodos e instanciam objetos (ex.: `PessoaFisica`, `PessoaJuridica`, `ContaEspecial`, `ContaPoupanca`).

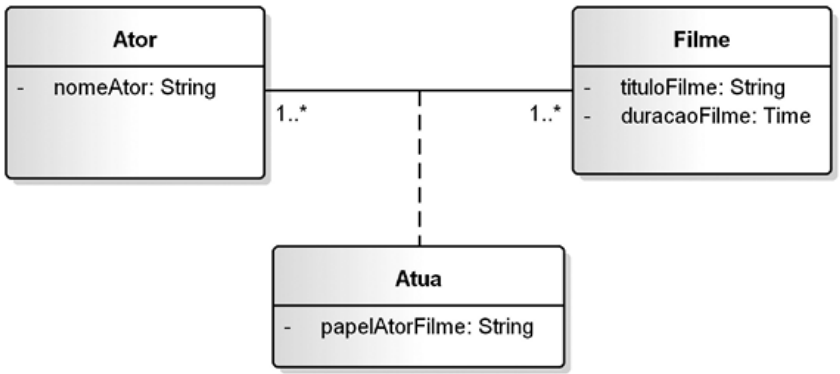
Hierarquia Bancária (Figura 4.12):



Classe Associativa vs. Classe Intermediária

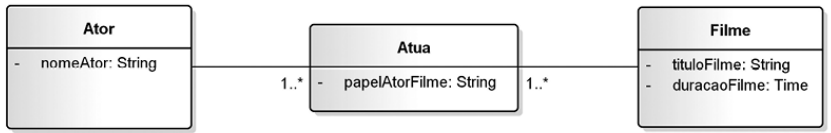
1. Classe Associativa (Fig. 4.13):

- Ligada por linha tracejada ao meio de uma associação (geralmente $*$ para $*$).
- Armazena atributos que pertencem **ao vínculo** (ex.: `papelAtorFilme`).
- **Restrição:** Admite apenas **uma única ocorrência** para cada par de objetos (a, f).



2. Classe Intermediária (Fig. 4.14):

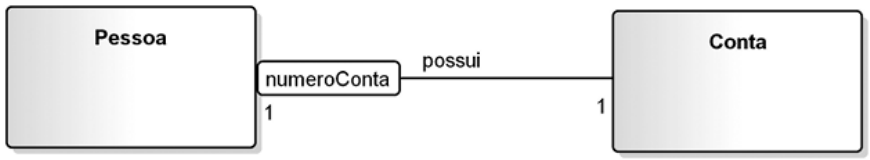
- Quando o mesmo par de objetos pode se relacionar **múltiplas vezes** (ex.: um ator interpreta dois papéis no mesmo filme).
- Substitui-se a ligação tracejada por uma **classe intermediária normal** com associações 1 para 1..*.



Associação Qualificada e Relacionamento de Dependência

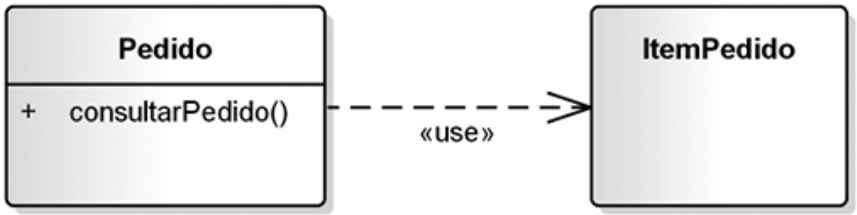
Associação Qualificada (4.15):

- Utiliza um atributo chave (qualificador em pequeno retângulo, ex.: `numeroConta`) para indexar uma coleção.
- Reduz a multiplicidade de `1..*` para `1` na busca direta de uma instância.



Dependência (- - ->) (4.16):

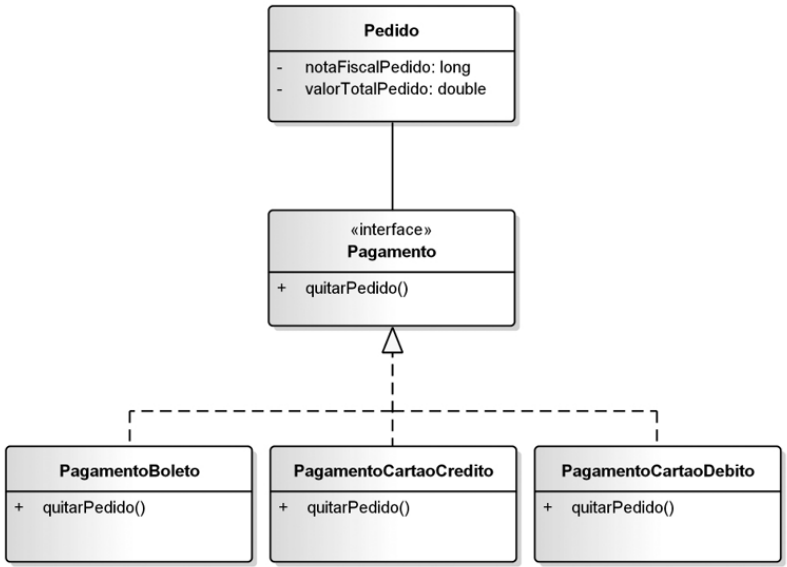
- Relação cliente/fornecedor fraca e transitória (ex.: classe `Pedido` recebe ou usa um objeto `Pagamento` como parâmetro de método).
- Linha tracejada com seta aberta apontando para o fornecedor.



Realização e Interfaces («interface»)

Realização de Contrato (Fig. 4.18):

- Uma interface define assinaturas de operações sem implementá-las (`+quitarPedido()`).
- Classes concretas (`PagamentoBoleto`, `PagamentoCartaoCredito`) **realizam** a interface (linha tracejada com triângulo vazio).



Notação Pirulito/Soquete (4.22):

- **Interface Fornecida (—○):** Círculo fechado (serviço provido pela classe).
- **Interface Requerida (⊂):** Semicírculo (serviço consumido pela classe através de uma Porta).



Restrições, Atributos Estáticos e Linguagem OCL

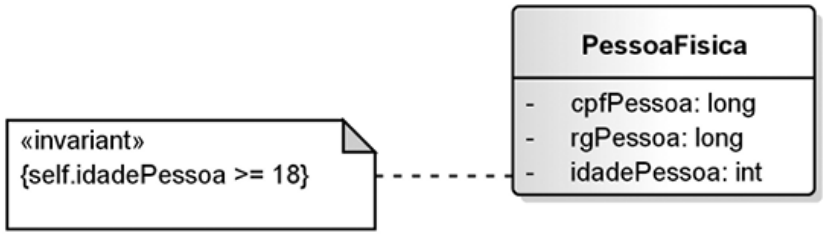
Propriedades e Restrições entre Chaves:

- `{readOnly}` **(4.28):** Atributo somente leitura após criação (ex.: `numeroConta`).
- **Membro Estático (4.29):** Grafado sublinhado; seu valor é compartilhado por toda a classe (ex.: `tomLigacao`).
- `{ordered}`, `{unique}`, `{xor}`: Regras sobre coleções e associações.

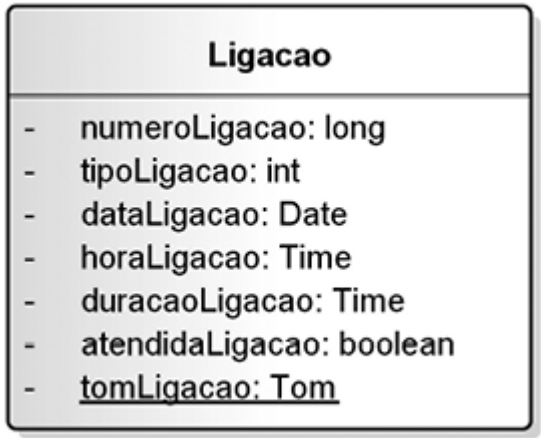
OCL (Object Constraint Language) (4.25):

- Especifica invariantes formais sem ambiguidade: `context PessoaFisica inv: self.idade >= 18`.

Restrição OCL (Fig. 4.25):



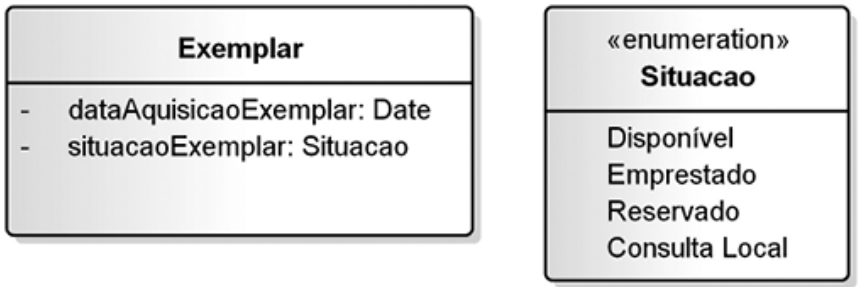
Atributo Estático Sublinhado (Fig. 4.29):



Estereótipos de Classes: Enumeração e Padrão MVC/ECB

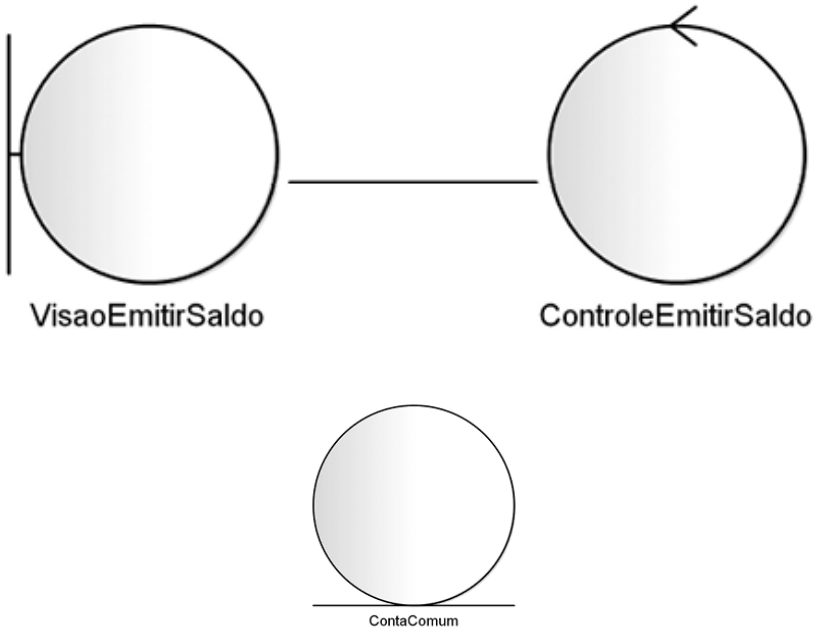
1. «enumeration» (Fig. 4.38):

- Lista o domínio fechado de valores literais válidos para um atributo (ex.: `SituacaoExemplar`: `disponivel`, `emprestado`, `atrasado`).



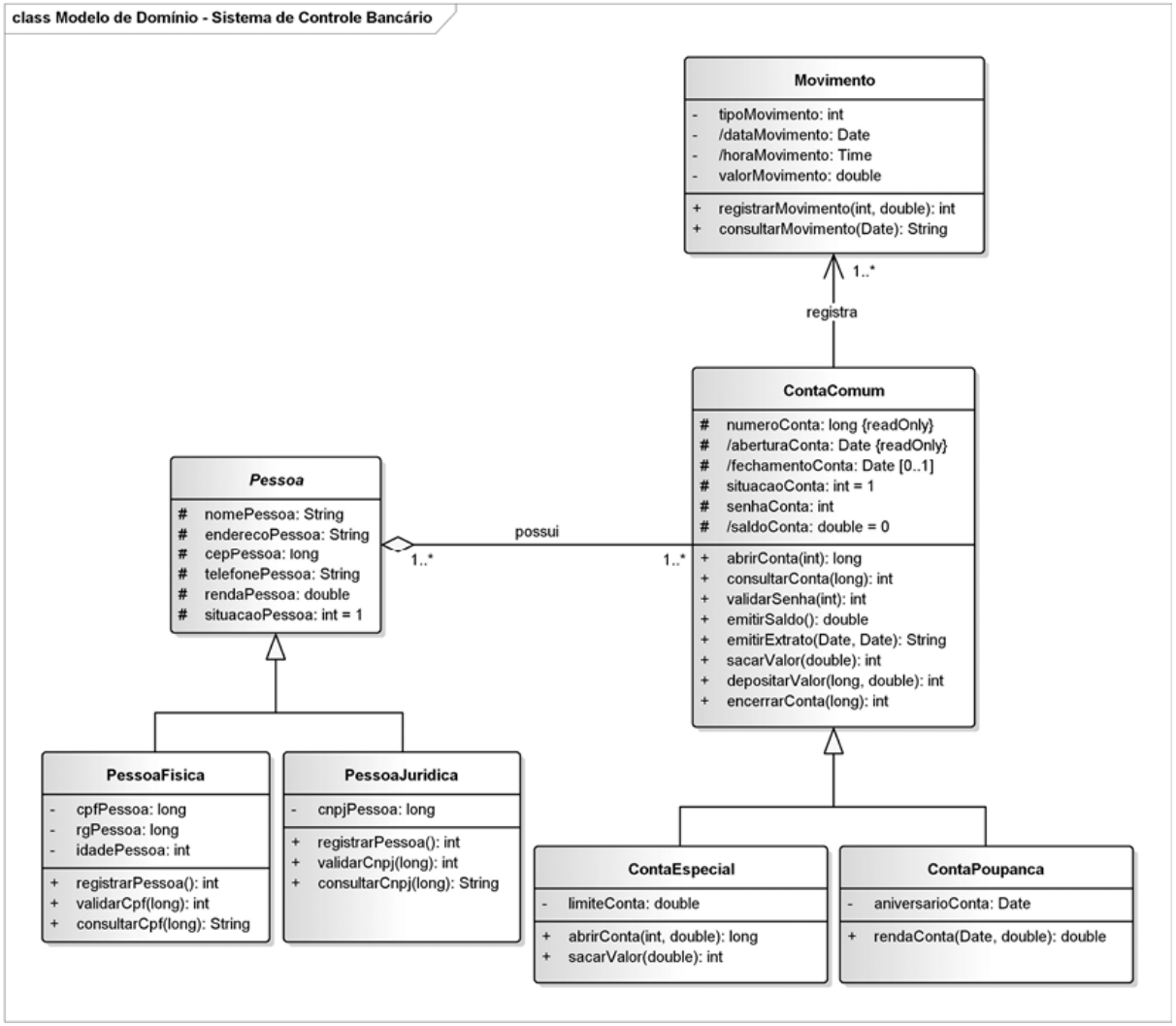
2. Camadas MVC / ECB (4.44 e 4.47):

- «boundary» **(Fronteira/Visão):** Telas e interfaces com usuário;
- «control» **(Controle):** Coordena casos de uso;
- «entity» **(Entidade/Modelo):** Objetos de negócio persistentes.



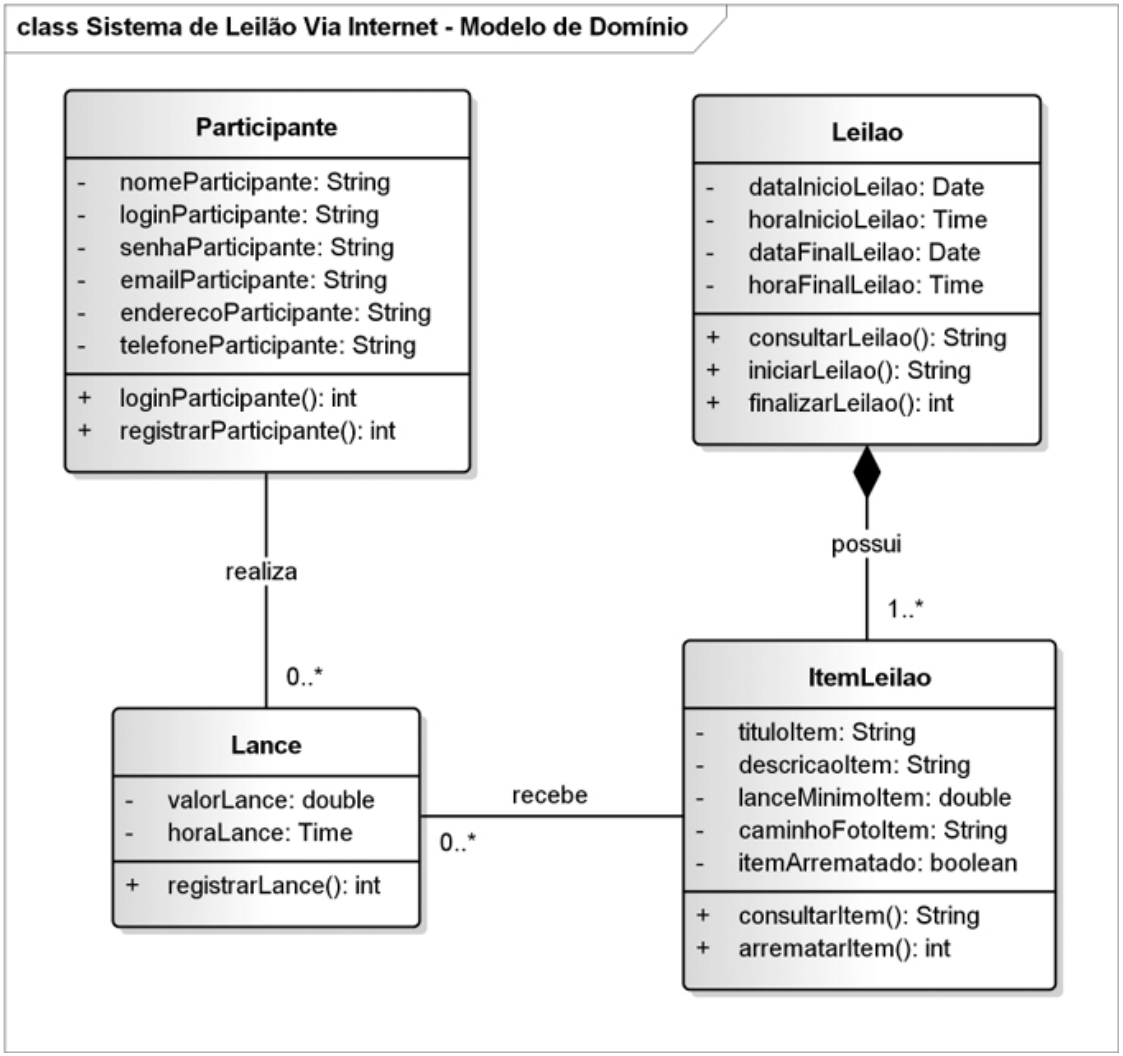
Estudo de Caso: Modelo de Domínio do Sistema Bancário

Diagrama de Classes - Modelo de Domínio (Figura 4.51 do Livro):



Reúne classe abstrata (*Pessoa*), herança, agregação (◊), navegabilidade, atributos derivados (/) e `{readOnly}`.

Modelo de Domínio para Leilão Online (Figura 4.72 do Livro):

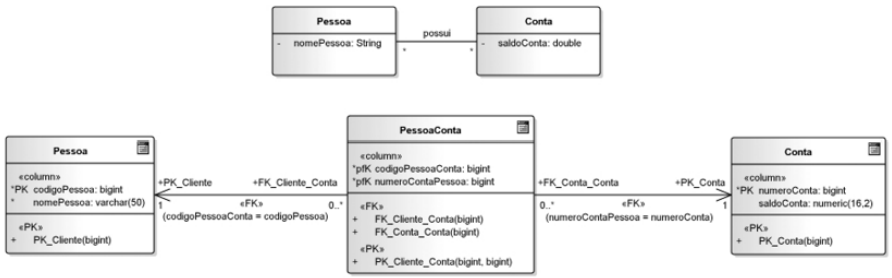


Observe a Composição (♦) entre `Leilao` e `ItemLeilao` e a classe intermediária `Lance` entre `Participante` e `ItemLeilao`.

Persistência, Mapeamento Relacional e Repository/DAO

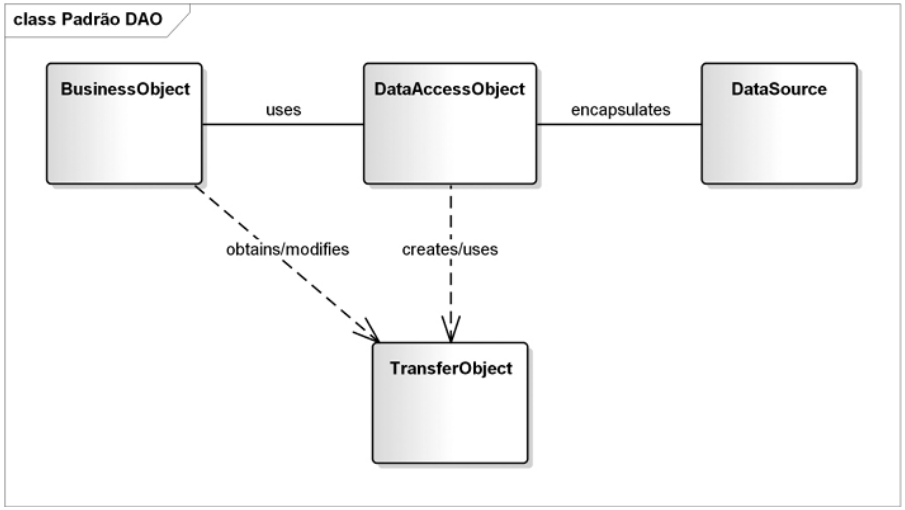
Mapeamento em Tabelas («Table»):

- **1:N (Fig. 4.59):** Chave estrangeira («FK») adicionada na tabela do lado N.
- **N:N (Fig. 4.60):** Criação de tabela asociativa intermediária com chaves estrangeiras compostas.
- **Herança (4.62–4.64):** Tabela única, uma tabela por classe concreta ou tabelas com «FK» para superclasse.



Isolamento de Persistência:

- **Padrão Repository (4.66):** Gerencia coleções de entidades em memória na camada de domínio.
- **Padrão DAO (4.68):** Encapsula comandos SQL e conexões com o SGBD na camada de persistência.



Mapeamento em Python: Composição e Realização

Trecho do arquivo `exemplo_classes_atividades.py` (Figuras 4.11 e 4.18):

```
class Edicao:
    def __init__(self, numero: int, volume: int, titulos_artigos: List[str]):
        self.numero_edicao = numero
        # Composicao forte (*--): Artigos instanciados e possuidos pela
            Edicao
        self.__artigos: List[Artigo] = [Artigo(t) for t in titulos_artigos]

class Pagamento(ABC):  # <<interface>> Pagamento (Figura 4.18)
    @abstractmethod
    def quitar_pedido(self, valor: float) -> str:
        pass

class PagamentoCartaoCredito(Pagamento):  # Realiza (implementa) Pagamento
    def quitar_pedido(self, valor: float) -> str:
        return f"Pagamento de R$ {valor:.2f} autorizado via Cartao de Credito
            ."
```

Diagrama de Atividades: Elementos Básicos (Cap. 10)

Finalidade do Diagrama de Atividades:

- Modela a lógica procedural de um algoritmo, método, workflow de negócios ou fluxo de um Caso de Uso.
- Equivale aos antigos fluxogramas evoluídos para suportar orientação a objetos e concorrência!

Nós Fundamentais (Fig. 10.2 a 10.5):

- **Nó Inicial (•):** Disparo da atividade;
- **Nó de Ação:** Passo atômico de execução (retângulo arredondado);
- **Fluxo de Controle ($\longrightarrow$):** Transição entre ações;
- **Nó de Final de Atividade ($\odot$):** Término global da atividade.

Nó Inicial e Ações (Fig. 10.4):



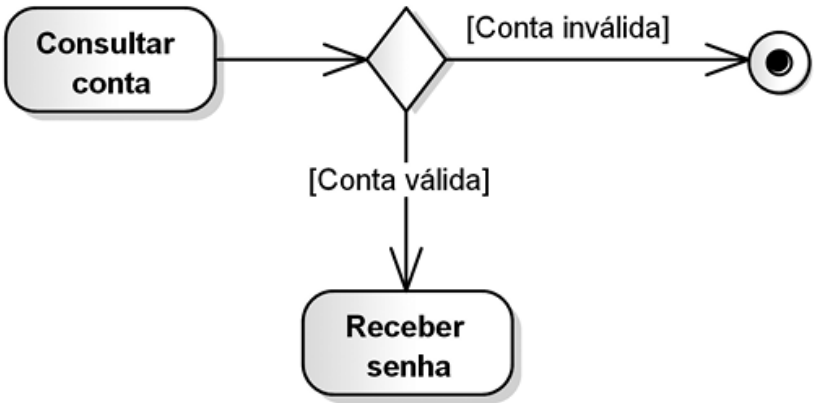
Nó de Final de Atividade (Fig. 10.5):



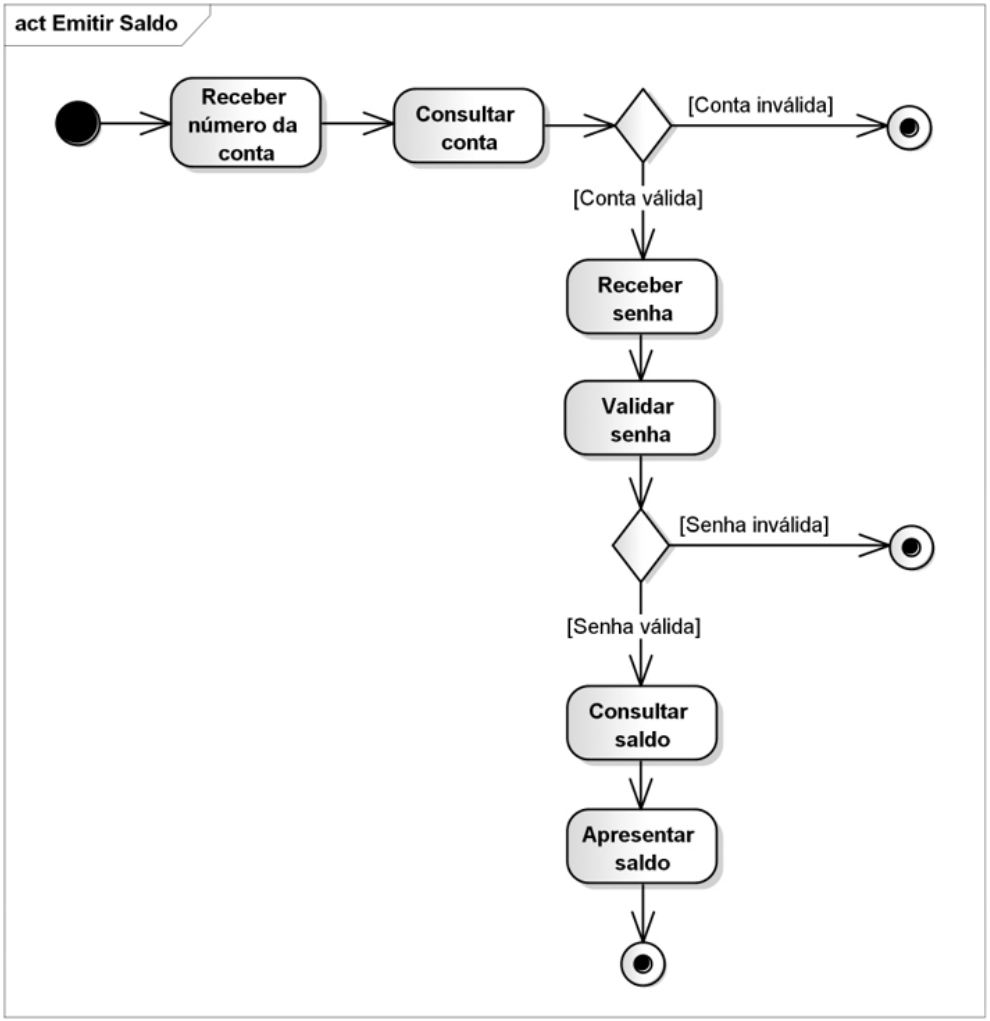
Nó de Decisão, Condições de Guarda e Fluxo Completo

Nó de Decisão / Fusão (10.6):

- Representado por um **losango (◊)**.
- Divide o fluxo em caminhos alternativos mutuamente exclusivos.
- Cada saída é rotulada por uma **Condição de Guarda entre colchetes** (ex.: `[Conta válida]` vs. `[Conta inválida]`).



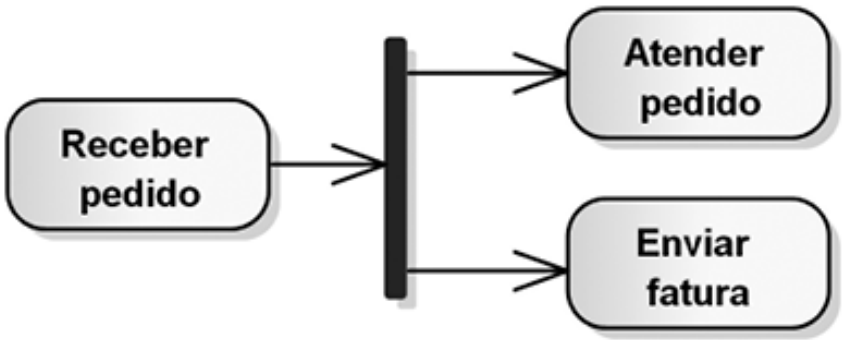
Atividade Emitir Saldo (Figura 10.7):



Paralelismo (Bifurcação/União) e Final de Fluxo

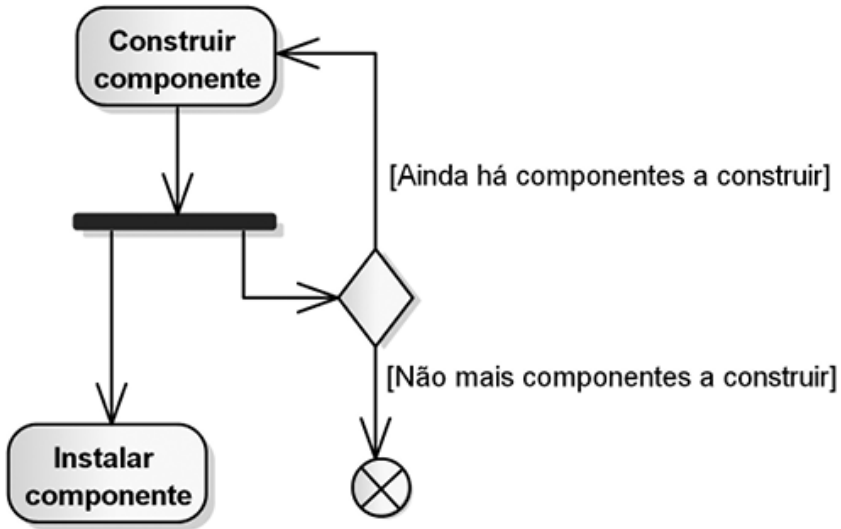
Barra de Bifurcação e União (10.8):

- **Bifurcação (*Fork*):** Barra espessa que divide 1 fluxo em 2 ou mais fluxos **concorrentes/paralelos**.
- **União (*Join*):** Sincroniza e aguarda a conclusão de todos os ramos paralelos antes de prosseguir.



Nó de Final de Fluxo (⊗) (10.9):

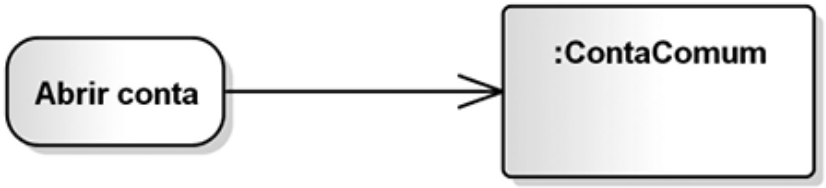
- Círculo contendo um **X**.
- Encerra **apenas o ramo paralelo local**, permitindo que as demais threads/ramos da atividade continuem executando!



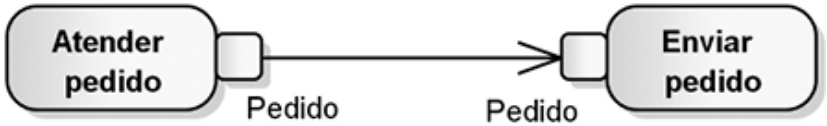
Fluxo de Objetos, Alfinetes (*Pins*) e DataStore

Nó de Objeto e Estado (10.10):

- Retângulo representando uma instância manipulada ou produzida por uma ação, podendo exibir seu estado entre colchetes (ex.: `ContaComum [Aberta]`).

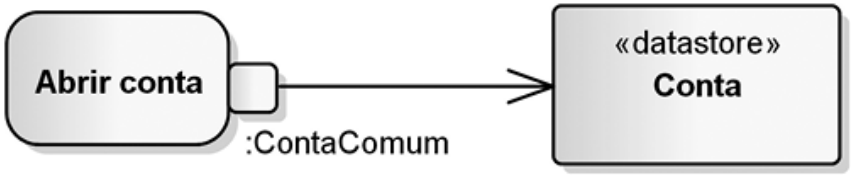


Alfinetes / *Pins* (10.11):



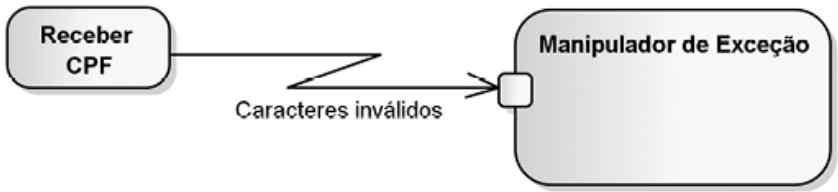
Repositório de Dados (10.13):

- Nó de objeto com estereótipo «`datastore`» que armazena dados de forma persistente durante a atividade.



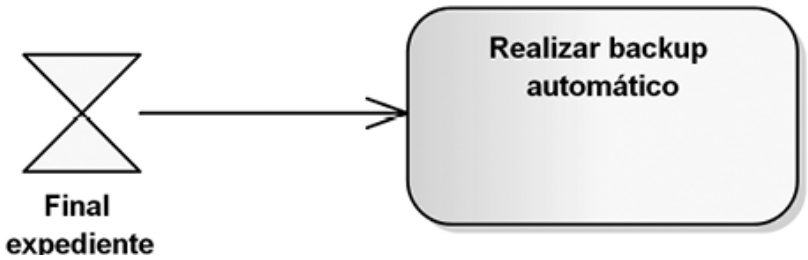
1. Fluxo de Interrupção / Exceção (10.14):

- Seta em formato de **raio (zigue-zague)** ligando a ação geradora ao manipulador de exceção.



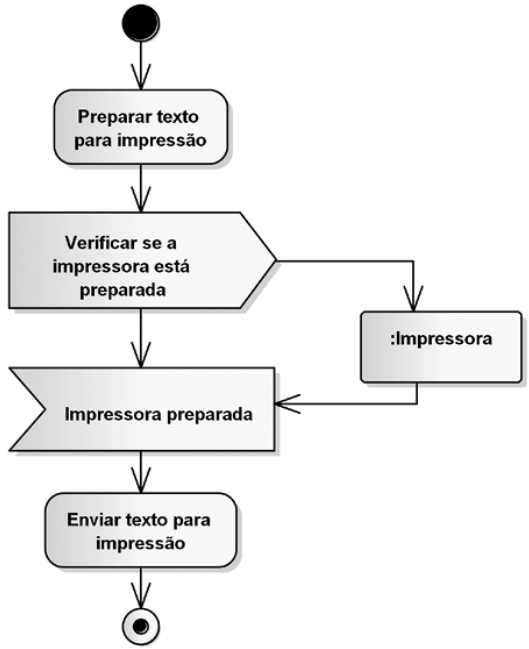
2. Evento de Tempo / *Timer* (10.16):

- Símbolo de ampulheta (disparo temporal ou *cron/trigger*).



3. Envio e Aceitação de Sinal (10.15):

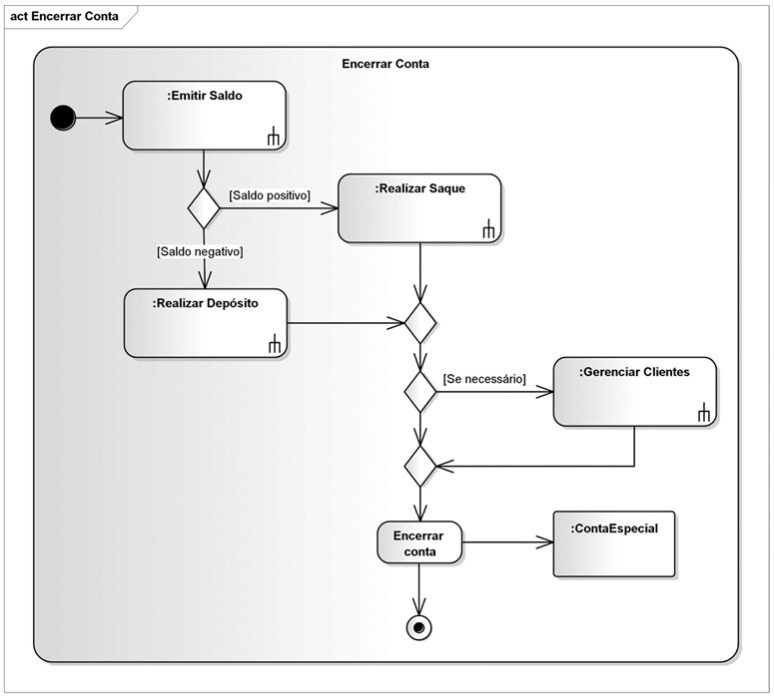
- **Envio de Sinal:** Polígono convexo (ponta para a direita);
- **Aceitação de Evento:** Polígono côncavo (reentrância à esquerda) que aguarda resposta assíncrona.



Chamada de Comportamento vs. Chamada de Operação

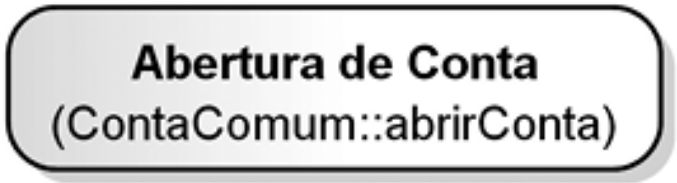
Ação de Chamada de Comportamento (10.17):

- Invoca outra **Atividade já modelada** (símbolo de **ancinho/tridente** no canto inferior direito).
- Permite modularizar diagramas extensos reaproveitando subatividades (*Consultar Conta*, *Realizar Saque*).



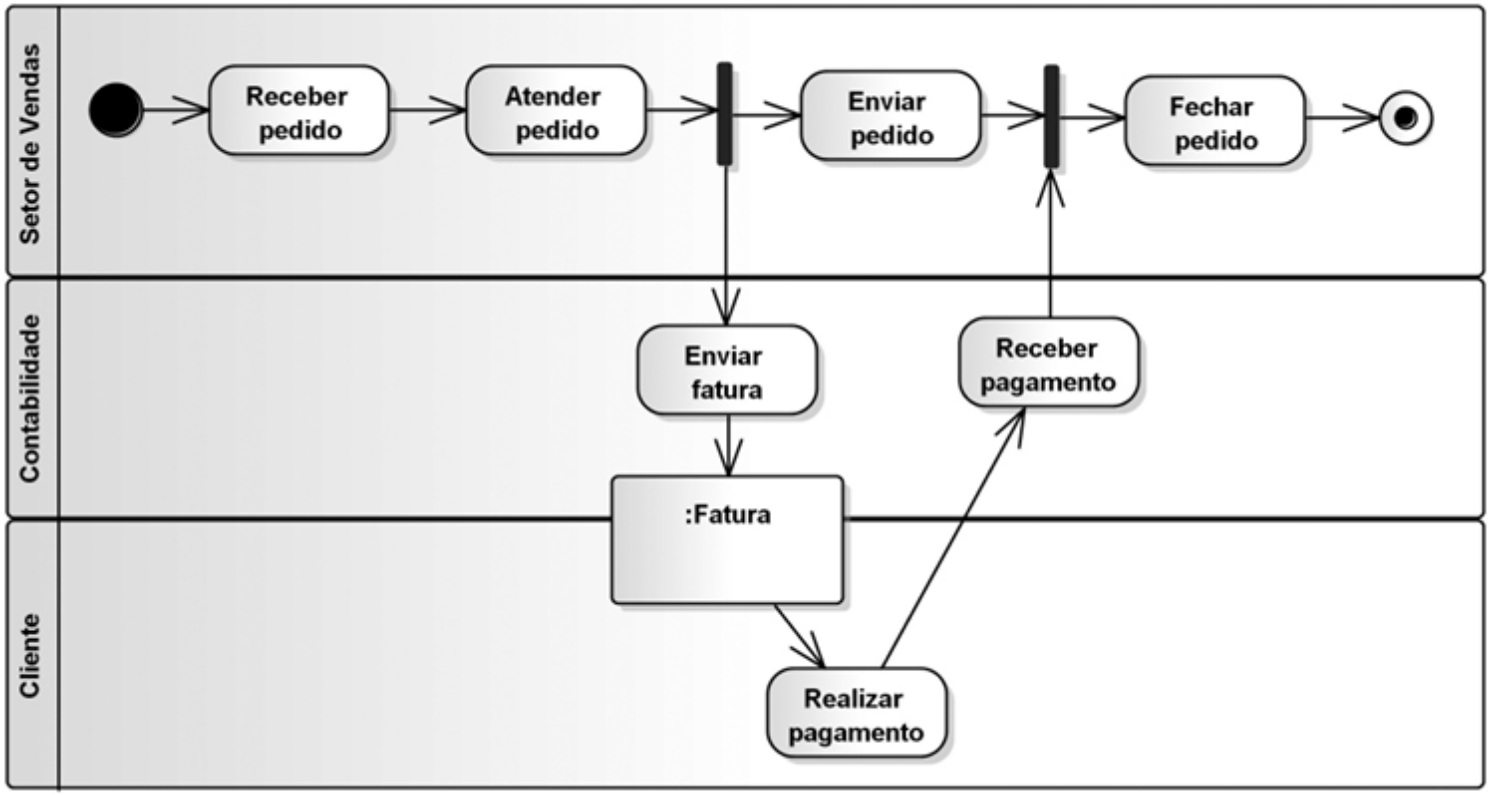
Chamada de Operação (10.18):

- Invoca diretamente um **método de uma classe** na notação `(Classe::metodo)`.



Partições de Atividade (*Swimlanes* / Raias de Natação)

Organização de Responsabilidades por Setor ou Ator (Figura 10.19):



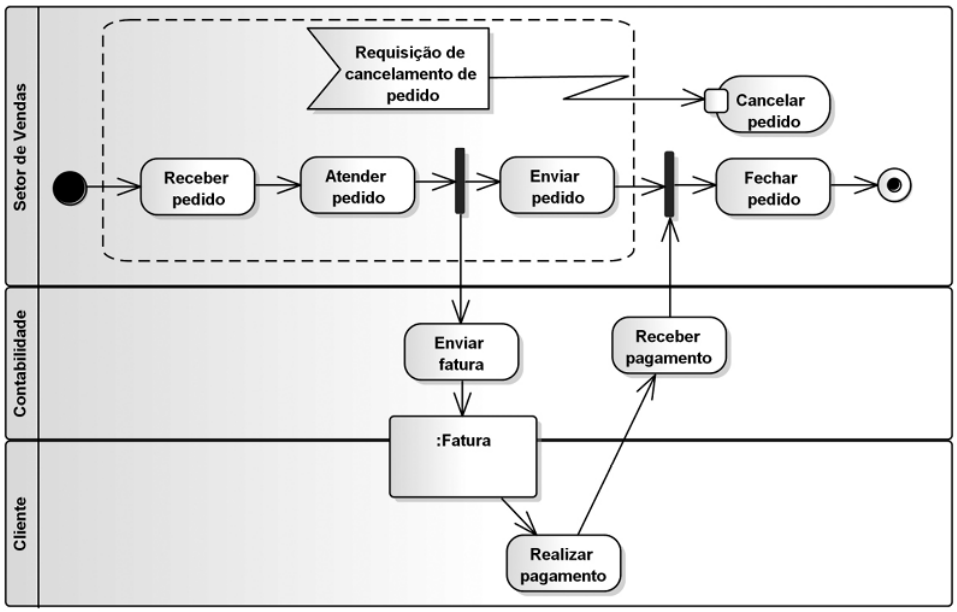
Importância Prática das Raias (*Swimlanes*)

Dividem visualmente a atividade em faixas horizontais ou verticais (**Setor de Vendas**, **Contabilidade**, **Cliente**), deixando explícito **quem é responsável** por executar cada ação do processo!

Região de Atividade Interrompível e Nós de Laço

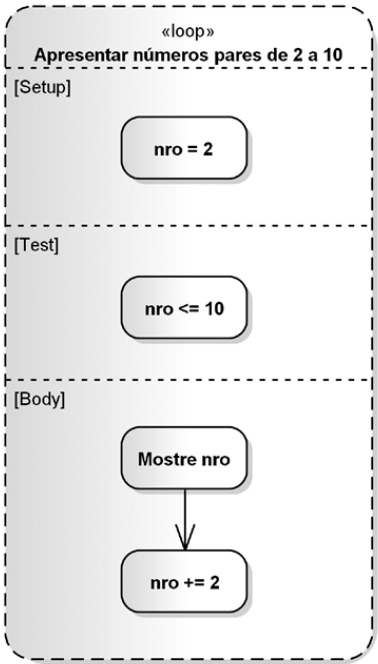
Região Interrompível (Fig. 10.20):

- Retângulo tracejado com bordas arredondadas envolvendo um grupo de ações.
- Se um evento de interrupção ocorrer (ex.: *Cancelar pedido*), um fluxo em raio aborta imediatamente todas as ações internas da região.



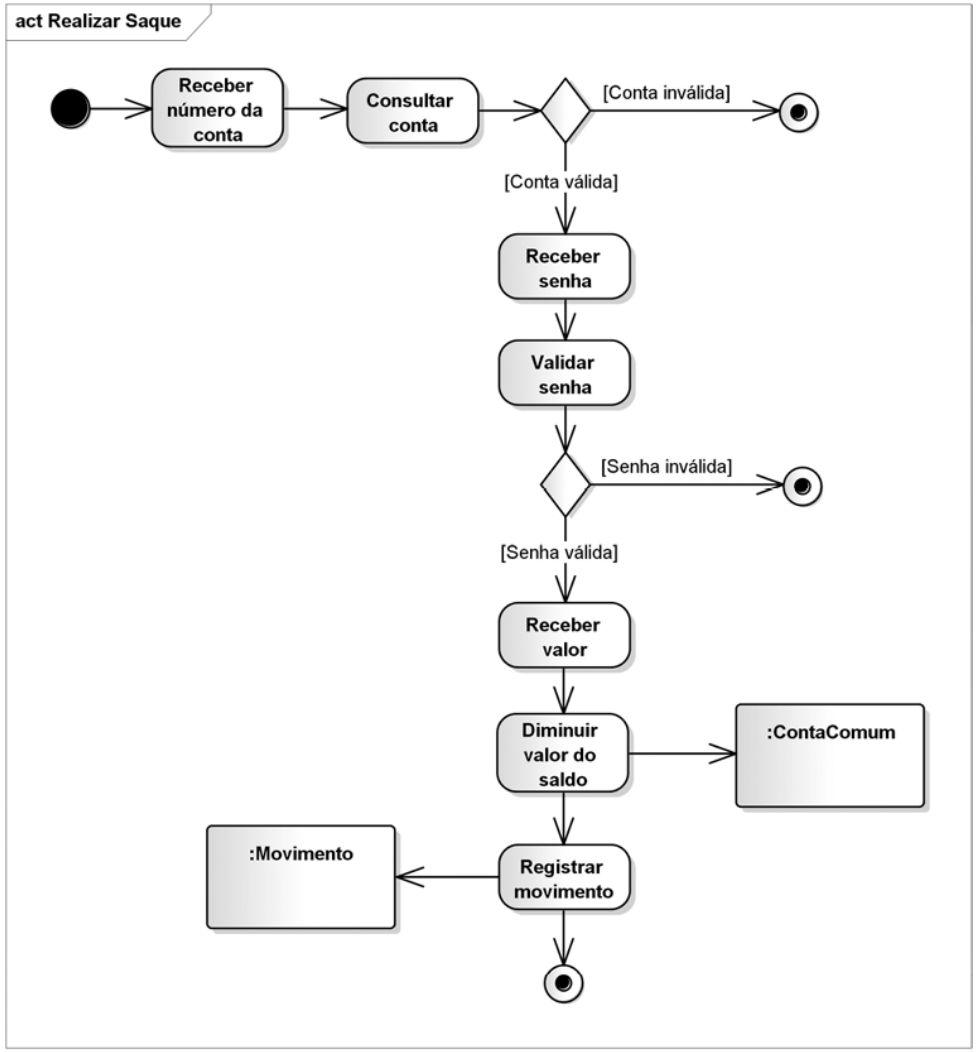
Nó Estruturado de Laço (10.23):

- Estereótipo «`loop node`» dividido em *Setup* (iniciação), *Teste* e *Corpo* iterativo.



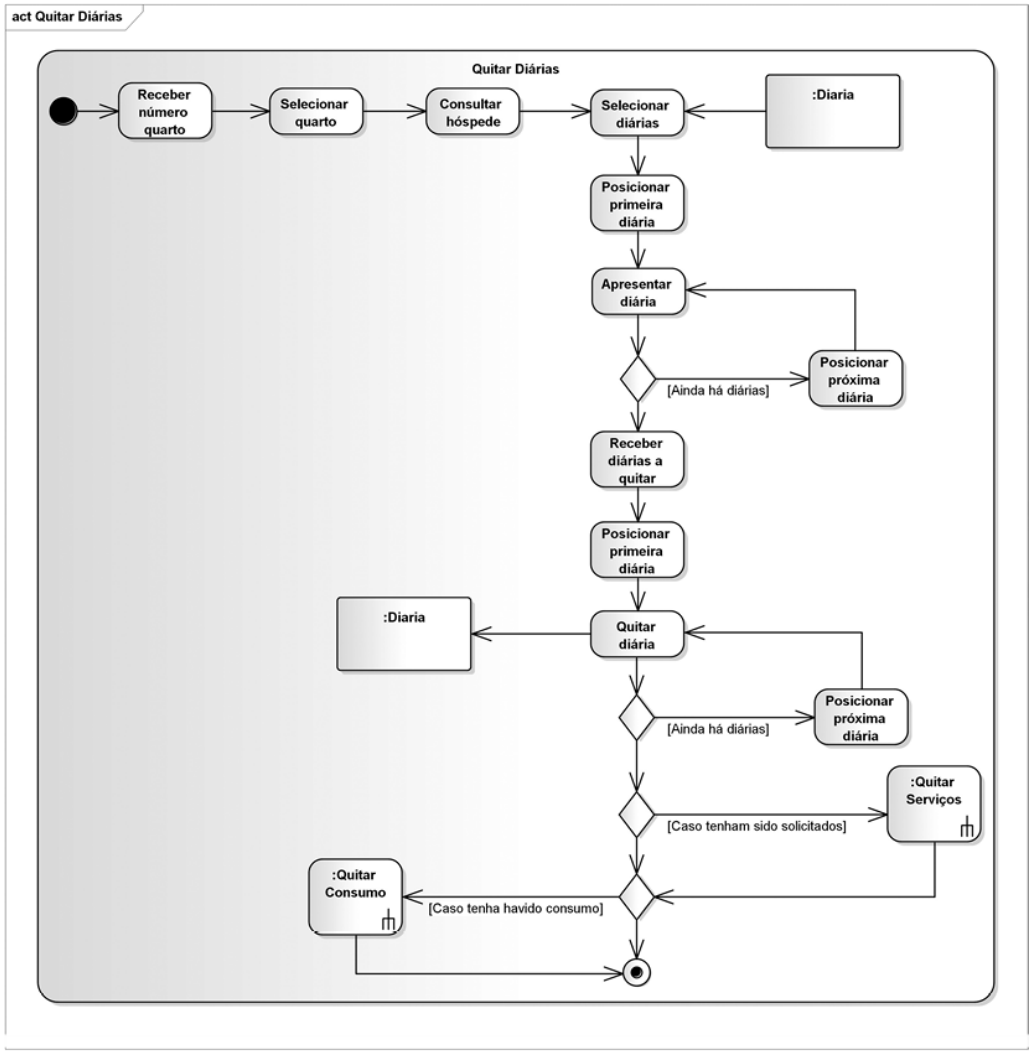
Estudo de Caso Bancário: Processo de Realizar Saque

Diagrama de Atividade Completo - Realizar Saque (Figura 10.28):



Combina decisões de validação de conta/senha/saldo, chamada de comportamento e atualização dos objetos `ContaComum`, `ContaEspecial` e `Movimento`.

Diagrama de Atividade - Sistema de Leilão Via Internet (Figura 10.32):



Demonstra laços de recepção de lances, controle de tempo de anúncio e transição de estado do objeto `ItemLeilao [Arrematado]`.

Atividade Prática: Estrutura e Fluxo do *TauáDelivery Web*

Continuidade do Projeto *TauáDelivery Web*

Na Aula 20 modelamos os Casos de Uso (**RF01 a RF13**). Agora projetaremos o **Diagrama de Classes de Domínio** e o **Diagrama de Atividades com Raias**!

Escopo do Diagrama de Classes:

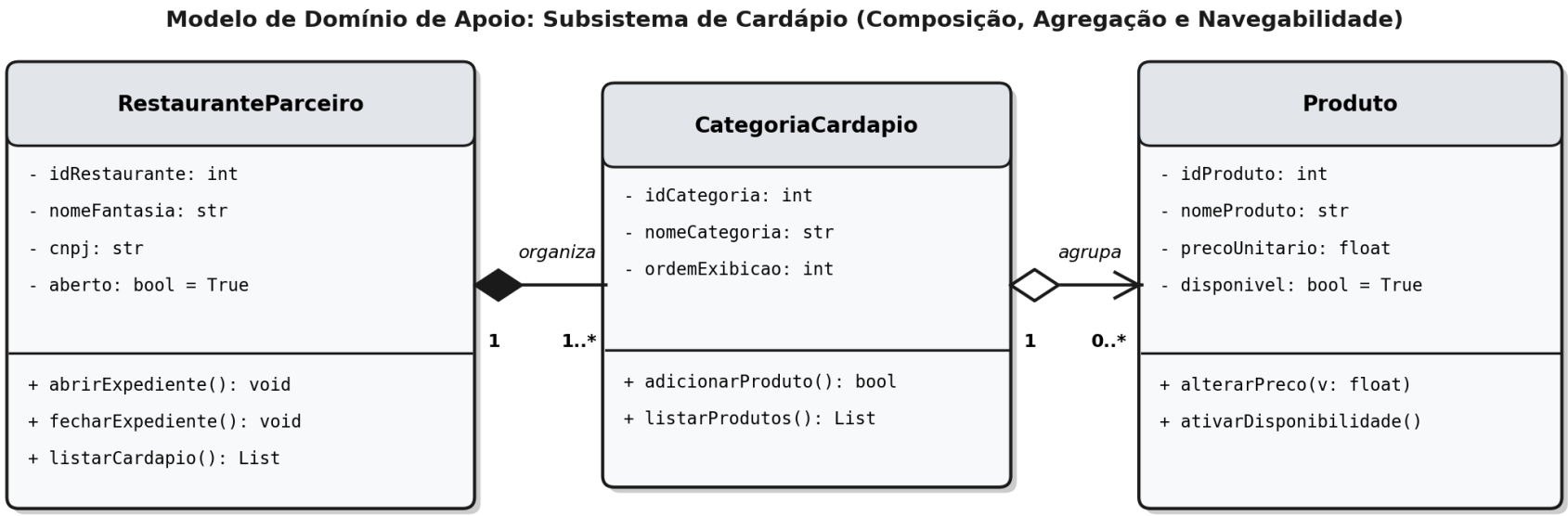
- Superclasse abstrata `UsuarioCadastrado` $\triangleleft$ `Cliente` e `Entregador`;
- Composição: `Pedido` $\blacklozenge \rightarrow$ `ItemPedido` $\rightarrow$ `Produto`;
- Enumeração «`enumeration`» `StatusPedido`;
- Interface «`interface`» `Pagamento` realizada por `PagamentoPix` e `PagamentoCartaoCredito`.

Escopo do Diagrama de Atividades:

- Processo **UC03 - Finalizar Pedido**;
- **3 Partições (Raias):** Cliente, Sistema TauáDelivery e Gateway de Pagamento;
- Decisões de autenticação e aprovação financeira;
- Paralelismo (*Fork/Join*) para notificar restaurante e emitir comprovante.

Exemplo de Apoio 1: Modelo de Classes do Cardápio

Subsistema de Cardápio (Modelo de Referência Resolvido):

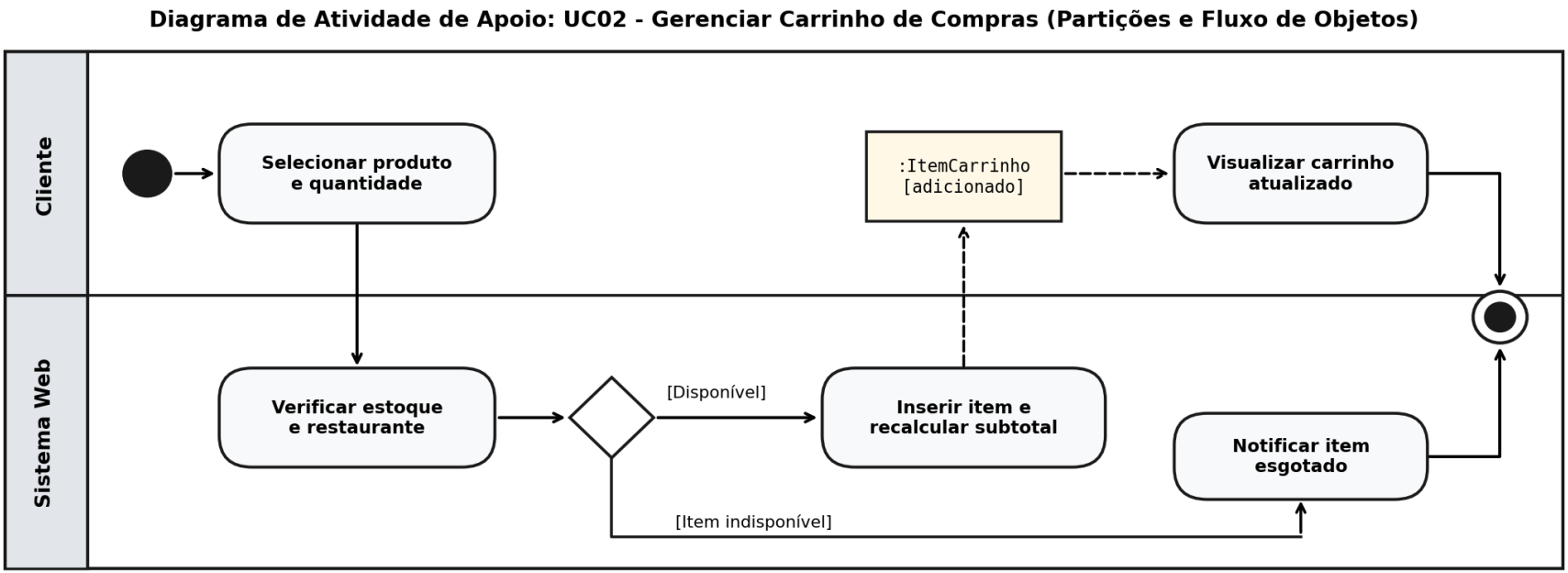


Análise Técnica do Exemplo

`RestauranteParceiro` compõe (♦) suas `CategoriaCardapio` (1 para `1..*`), enquanto cada categoria agrega (◊) e navega para os objetos `Produto` (1 para `0..*`).

Exemplo de Apoio 2: Diagrama de Atividades do Carrinho

Fluxo com Partições para UC02 - Gerenciar Carrinho (Modelo de Apoio):



Análise Técnica do Exemplo

Separa claramente as ações das raias **Cliente** e **Sistema Web**, aplicando Nó de Decisão com guardas `[Disponível]` / `[Item indisponível]` e Fluxo de Objetos `:ItemCarrinho [adicionado]`.

Roteiro de Entrega da Atividade Prática

Parte 1: Questões Conceituais (3,0 pts)

- Diferença de ciclo de vida entre Agregação (◊) e Composição (⧫);
- Classe Associativa vs. Classe Intermediária para avaliações de pedidos;
- Vantagem do polimorfismo com `«interface» Pagamento`;
- Nó de Decisão vs. Barra *Fork/Join* e Final de Atividade vs. Final de Fluxo.

Partes 2 e 3: Modelagem Visual (7,0 pts)

- **Parte 2 (4,0 pts):** Diagrama de Classes completo de Pedidos, Entregas e Pagamentos com multiplicidades, visibilidades e atributos derivados (/).
- **Parte 3 (3,0 pts):** Diagrama de Atividades com 3 raias para o processo *UC03 - Finalizar Pedido*.

Dica de Ouro

Consultem o arquivo `atividade_pratica.md` e executem `python3 exemplo_classes_atividades.py` para validar os conceitos em código!

Conclusão e Próximos Passos

- **O Diagrama de Classes materializa a arquitetura estática:** Define com rigor a hierarquia, o acoplamento todo-parte (agregação/composição), os contratos de interface e o mapeamento relacional.
- **O Diagrama de Atividades governa o fluxo dinâmico:** Transforma especificações textuais de casos de uso em fluxos visuais precisos com decisões, paralelismo e raias de responsabilidade.

Próxima Aula (Aula 22)

Diagramas de Interação e Máquina de Estados (Capítulos 7, 8 e 9): Diagrama de Sequência (troca cronológica de mensagens e *lifelines*), Diagrama de Comunicação e ciclo de vida de objetos com Máquina de Estados.